

Teil 3:

Architektur und Modellierung

Prof. Dr. Falk Howar

Automated Quality Assurance
Lehrstuhl für Software Engineering
Fakultät für Informatik
TU Dortmund



Architektur.

- Die 4+1 Sichten auf Architektur **aufzählen**, **erläutern** und damit eine Architektur **beschreiben**.



- Treiber für Architektur-Entscheidungen **verstehen**



- Qualitätskriterien auf Architekturen **anwenden**, um diese zu **vergleichen**



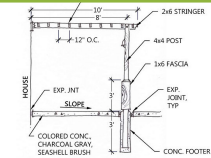
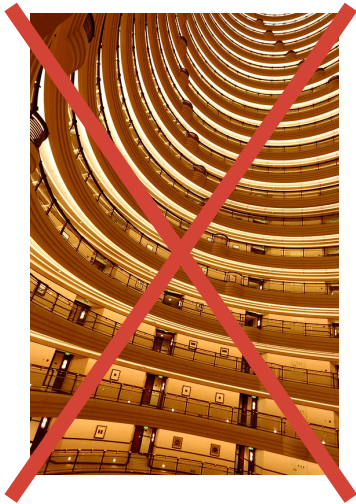
- Bekannte Architekturmuster **aufzählen** und **erläutern** sowie deren Eignung für einen Anwendungsfall **bewerten**



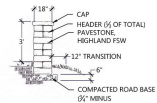
Agenda

1. Motivation
2. Begriffe und Definitionen
3. Treiber und Prinzipien
4. Dokumentation: 4+1 Sichten
5. Ausgewählte Architekturmuster
6. Qualität von Software Architektur
7. Zusammenfassung
8. Literatur und Standards

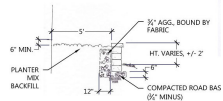
Analogien "Architektur"



① WOOD ARBOR



② FREE STANDING WALL



③ RETAINING WALL

<https://flic.kr/p/9FbvPx> (CC BY)

<https://flic.kr/p/9mafXs> (CC BY)

Analogien “Architektur”

Dokumentation:

- *“Andere Disziplinen, wie Maschinenbau, Immobilienarchitektur und Elektrotechnik, haben ungeheuer von der Standardisierung ihrer jeweiligen Pläne profitiert.”*
- *“Heute kann jeder Ingenieur die Konstruktionszeichnungen seiner Berufskollegen verstehen und weiterentwickeln.”*

Konstruktionsprinzipien unterstützen Ziele:

- Wir wissen aus Erfahrung und Versuchen, wie eine Brücke konstruiert sein muss, um bestimmte Belastungen auszuhalten
- Wir haben Muster, nach denen wir Verkehrskreuzungen und Autobahnkreuze kosten-effizient, sicher, effizient, usw. gestalten.

Zitate: G. Starke, P. Hruschka: Software-Architektur kompakt — angemessen und zielorientiert (2. Auflage), 2011

Beispiel: Remote Home Control



Ideen für Architektur?

Abschnitt	Inhalt
Bezeichner	UC-1
Name	Gerät schalten
Autoren	F. Howar
Priorität	...
Quelle	...
Verantwortlich	...
Kurz-Beschreibung	Nutzer schaltet ein Gerät von unterwegs ein.
	Nutzer wünscht, ein Gerät zu schalten.
	App installiert und Gerät registriert.
Voraussetzung	
Nachbedingung	
Ergebnis	
Hauptszenario	App öffnen, Gerät aus Übersicht wählen, Gerät schalten
Alternativ-Szenarien	...
Ausnahme-Szenarien	...
Einzelne Anforderungen	REQ-F-10, ...
Qualitäten	REQ-Q-1, REQ-Q-2, ...

<https://flic.kr/p/oStisy> (CC BY 2.0)

Beispiel: Remote Home Control

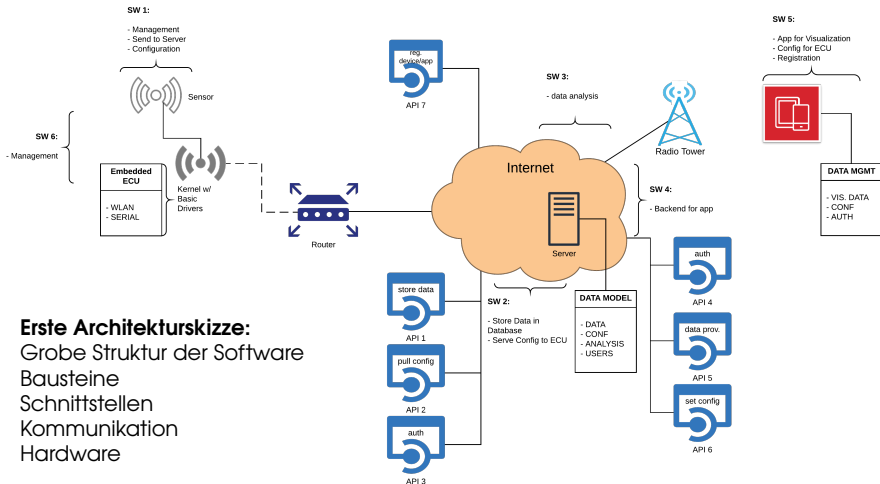
Fragen:

- Wie werden Systeme im Haus angebunden?
- Wie kommuniziert die App mit den Systemen?
- Wie erhält ein Nutzer Benachrichtigungen?

Herausforderungen und Ziele:

- Verteiltes System
- Sicherheit

Beispiel: Remote Home Control



Erste Architekturskizze:

Grobe Struktur der Software

Bausteine

Schnittstellen

Kommunikation

Hardware

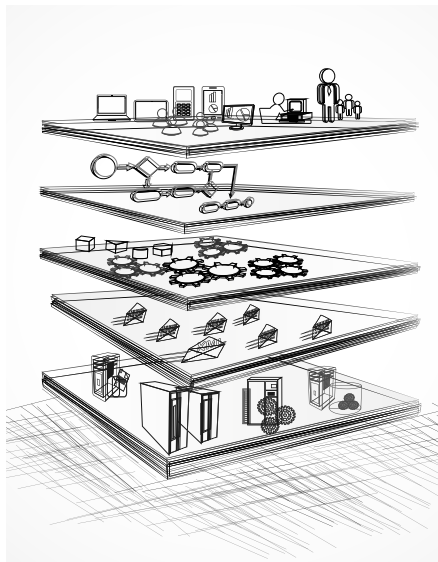
Software Architektur

Ziele:

- Grobe Struktur, die bestimmte (nicht-funktionale) Anforderungen erfüllt
- Dokumentation der groben Struktur

Aspekte:

- Logische Struktur (z.B. Teil-Systeme, Dienste, Komponenten, Schnittstellen, ...)
- Dynamische Aspekte: Abläufe im System, Kommunikation zwischen Teilen
- Verteilung von Software auf Hardware
- Struktur für die Entwicklung (Code, Artefakte)



Quelle: Shutterstock

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Treiber und Prinzipien
4. Dokumentation: 4+1 Sichten
5. Ausgewählte Architekturmuster
6. Qualität von Software Architektur
7. Zusammenfassung
8. Literatur und Standards

Software Architektur

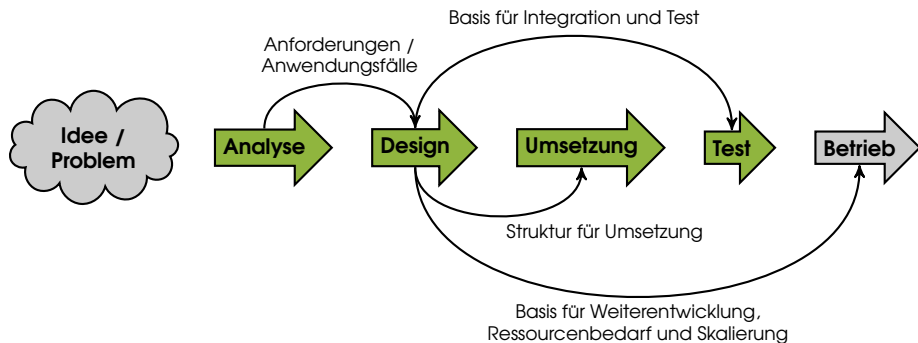
Definition (Software Architektur)

- ① Die Architektur eines Systems beschreibt die Strukturen des Systems, dessen *Bausteine*, *Schnittstellen* und deren Zusammenspiel.
- ② Die Architektur besteht aus Plänen und enthält meistens Vorschriften oder Hinweise, wie das System erstellt oder zusammengebaut werden sollte.

Als Plan unterstützt die Architektur ...

- die Herstellung (hier: Programmierung und Test)
- die Weiterentwicklung und den Einsatz des Systems (hier: Betrieb)

Architektur im Entwicklungsprozess



Bausteine und Schnittstellen

Definition (Bausteine)

Bausteine sind unabhängige, ersetzbare **Teile** mit wohldefinierten Aufgaben und **Schnittstellen** (Interfaces), welche inneren Zustand und Verhalten **kapseln**.

Beispiele für Bausteine:

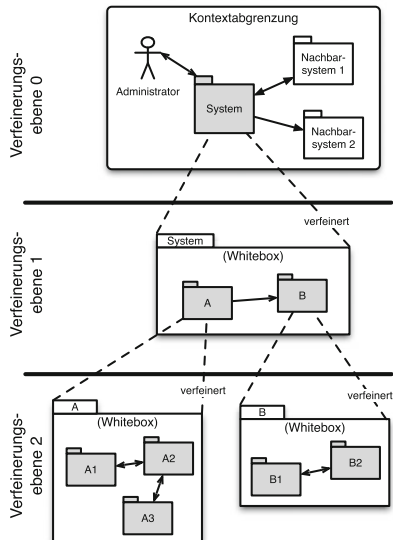
- Klassen
- Komponenten
- Dienste
- Teil-Systeme

Definition (Schnittstellen)

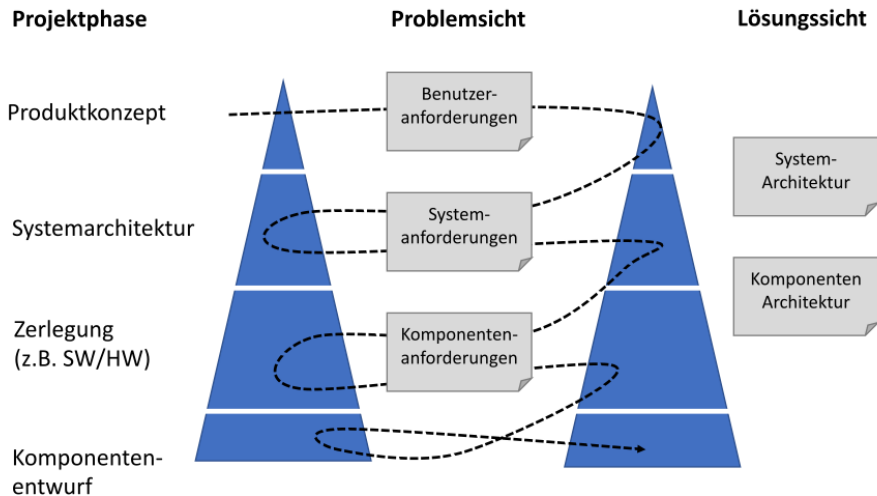
Schnittstellen sind definierte Interaktionspunkte (= Methoden, Datenformat und Protokoll) zwischen Teilen eines Softwaresystems.

Hierarchische Komposition und Präzisierung

- Software-Architekturen bestehen aus Bausteinen
- Bausteine tragen Verantwortung und kommunizieren über Schnittstellen
- **Blackboxes** sind nur durch ihre Schnittstellen und Funktionen charakterisiert. Sie verstecken ihr gesamtes Innenleben.
- **Whiteboxes** präzisieren Blackboxes durch das Angeben innerer Struktur.



Ko-Entwicklung von Architektur und Anforderungen



nach Ebert, 2014: Systematisches Requirements Engineering, 5. Auflage

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Treiber und Prinzipien
4. Dokumentation: 4+1 Sichten
5. Ausgewählte Architekturmuster
6. Qualität von Software Architektur
7. Zusammenfassung
8. Literatur und Standards

Treiber von Architektur Entscheidungen

Ziele:

- Architektur Entscheidungen sind getrieben von Zielen
- Es ist wichtig, diese Ziele festzulegen und zu priorisieren
- Verschiedene Entscheidungen zahlen unterschiedlich auf Ziele ein

Herausforderungen:

- Architektur Entscheidungen berücksichtigen Herausforderungen
- Herausforderungen sind teilweise spezifisch für Anwendungsszenarien (z.B. Verteilte Systeme)

Prinzipien:

- Prinzipien adressieren Herausforderungen
- Prinzipien sind Basis für Entscheidungen

Beispiele für Herausforderungen und Ziele

Entwicklung

- Modularität
- Flexibilität
- Einfache Releases
- Testbarkeit
- Verständlichkeit
- Einfache Releases
- ...

Betrieb

- Performanz
- Skalierbarkeit
- Sicherheit
- Stabilität
- Resilienz
- ...

Weitere

- Wiederverwendbarkeit
- Attraktivität
- ...

Herausforderung Struktur: Big Ball of Mud Anti-Pattern

- Big Ball of Mud = System, das keine erkennbare Softwarearchitektur besitzt

- **Gründe:**

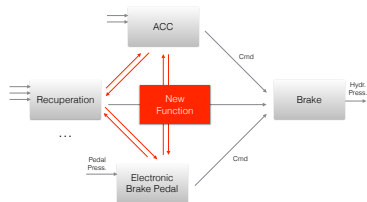
- ungenügende Erfahrung, fehlendes Bewusstsein für Softwarearchitektur
- Erosion

- **Konsequenzen:**

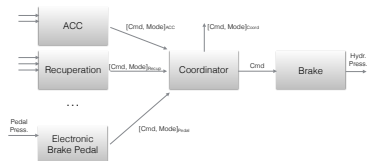
- Mangelnde Wartbarkeit
- Teure Releases
- Minimale Erweiterungen nur mit massivem finanziellen Aufwand

Beispiel:

Koordination verschiedener Assistenzsysteme



VS.



Herausforderung Entwicklung: Projekte und Abhängigkeiten

- Große Projekte werden nicht in einem Software Modul entwickelt
- **Gründe:**
 - Wiederverwendbarkeit
 - Skalierung der Entwicklung
 - Einfacher Austausch von Teilen
- **Konsequenzen:**
 - Abhängigkeiten zwischen Modulen

```
<groupId>de.learnlib</groupId>
<artifactId>learnlib-parent</artifactId>
<version>0.15.0-SNAPSHOT</version>
<packaging>pom</packaging>

<name>LearnLib</name>

<modules>
  <module>algorithms</module>
  <module>api</module>
  <module>archetypes</module>
  <module>build-parent</module>
  <module>build-tools</module>
  <module>commons</module>
  <module>datastructures</module>
  <module>distribution</module>
  <module>drivers</module>
  <module>oracles</module>
  <module>test-support</module>
</modules>

<dependencyManagement>
  <dependencies>
    <!-- algorithms -->
    <dependency>
      <groupId>de.learnlib</groupId>
      <artifactId>learnlib-algorithms-parent</artifactId>
      <version>${project.version}</version>
      <type>pom</type>
    </dependency>
    <dependency>
      <groupId>de.learnlib</groupId>
      <artifactId>learnlib-algorithms-active</artifactId>
      <version>${project.version}</version>
      <type>pom</type>
    </dependency>
    <dependency>
      <groupId>de.learnlib</groupId>
      <artifactId>learnlib-adt</artifactId>
      <version>${project.version}</version>
    </dependency>
    <dependency>
      <groupId>de.learnlib</groupId>
      <artifactId>learnlib-dhc</artifactId>
      <version>${project.version}</version>
    </dependency>
    <dependency>
      <groupId>de.learnlib</groupId>
      <artifactId>learnlib-discrimination-t</artifactId>
      <version>${project.version}</version>
    </dependency>
  </dependencies>
</dependencyManagement>
```

Beispiel: Maven POM Projekt LearnLib (Ausschnitt)

SOLID Prinzipien

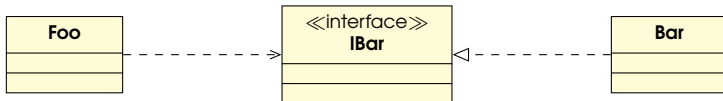
- **Single Responsibility:** Jeder Baustein, jede Schnittstelle hat genau eine Aufgabe (und damit nur eine Quelle für Änderungen) ⇒ Modularität
- **Open-Closed:** Die Architektur soll neue Funktionalität einfach erlauben (offen) aber vor Veränderungen schützen (geschlossen) ⇒ Stabilität
- **Liskov Substitution:** Zwei Komponenten mit der gleichen Schnittstelle müssen austauschbar sein ⇒ Kompositionalität
- **Interface Segregation:** Schnittstellen sollen so klein und spezifisch wie möglich sein ⇒ So wenig Abhängigkeiten wie möglich
- **Dependency Inversion:** High-Level Module sollen nicht von Low-Level Modulen abhängen ⇒ Einfaches Ändern von Details

Beispiel: Open-Closed

Schlechte Architektur:



Bessere Architektur:



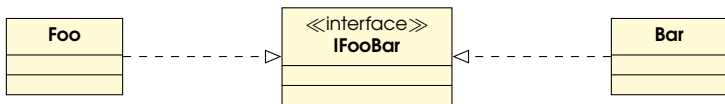
Bar kann einfach gegen Alternativen ausgetauscht werden, die IBar erfüllen

Beispiel: Liskov Substitution

Schlechte Architektur:



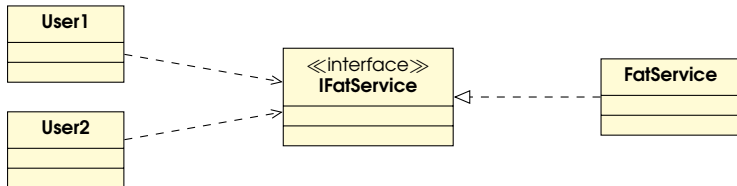
Bessere Architektur:



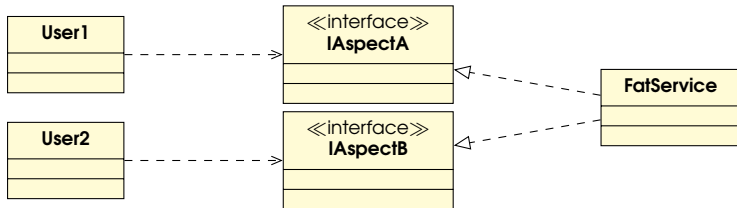
Foo und Bar implementieren beide IFooBar. In der schlechteren Variante verlässt sich Bar evtl. riskanterweise auf Teile der Implementierung von Foo.

Beispiel: Interface Segregation

Schlechte Architektur:



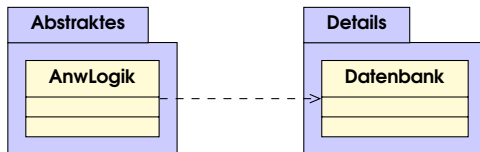
Bessere Architektur:



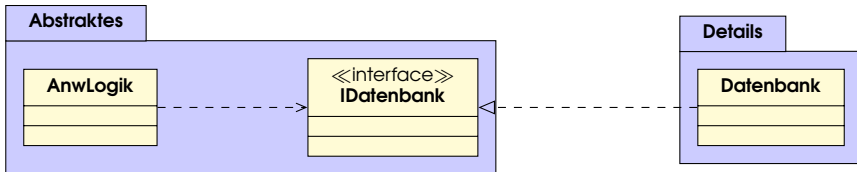
Weniger Abhängigkeiten für User1 und User2

Beispiel: Dependency Inversion

Schlechte Architektur:

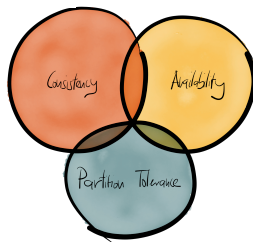


Bessere Architektur:



Details können sich ändern, ohne dass high-level Funktionalität angepasst werden muss

Herausforderung Verteilung: CAP-Theorem



"in larger distributed-scale systems, network partitions are a given; therefore, consistency and availability cannot be achieved at the same time."

Prinzipien / Muster zur Lösung → siehe Architekturmuster

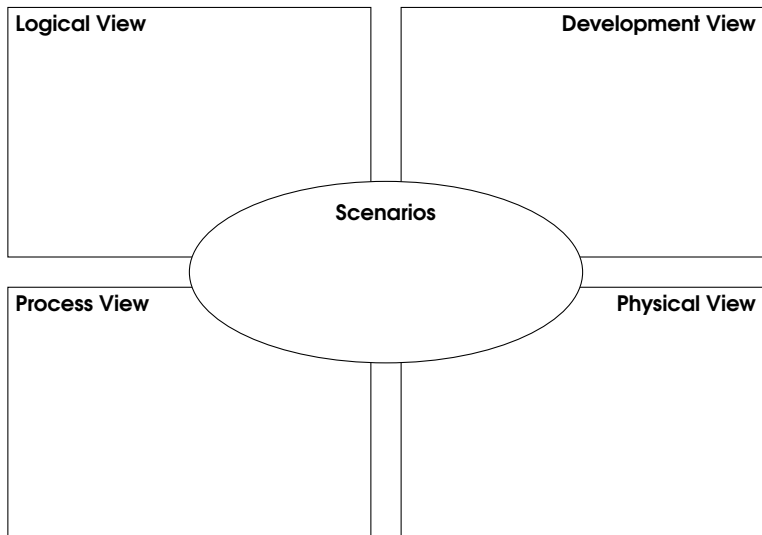
Agenda

1. Motivation
2. Begriffe und Definitionen
3. Treiber und Prinzipien
- 4. Dokumentation: 4+1 Sichten**
5. Ausgewählte Architekturmuster
6. Qualität von Software Architektur
7. Zusammenfassung
8. Literatur und Standards

Dokumentation von Architektur

- Aus den Zielen schrittweise operationelle **Szenarien** abzuleiten und die Architektur gegen diese Szenarien zu prüfen.
- Zur frühzeitigen Identifikation möglicher Risiken oder Probleme sollte die Dokumentation von Software-Architekturen die **Prüfbarkeit** der gewählten Lösung **hinsichtlich der an das System gestellten Anforderungen** unterstützen.
- **Unterschiedliche Architektursichten** ermöglichen durch Walkthroughs die Prüfung funktionaler und nicht-funktionaler Anforderungen anhand von Bewertungs- oder Prüfscenarien.

4+1 Sichten auf eine Architektur



Szenarios

- Anwendungsfälle aus Anforderungsdokumenten
- Grundlage für Prüfung der Architektur:
 - Beschreibung präzise und vollständig?
 - Werden Ziele erreicht?
- **Inhalt:**
 - Abläufe im System für bestimmte Anwendungsfälle
- **Diagramme:**
 - Anwendungsfalldiagramme
 - Textuelle Beschreibung von Anwendungsfällen

Logical View (Funktionale Sicht bzw. Bausteinsicht)

- **Kernstück der Software-Architektur, entspricht dem Grundrissplan eines Hauses**

- Zeigt statische Struktur des Systems
- Beschreibt wie das System aus Einzelteilen konstruiert ist

- **Inhalt:**

- Aufbau aus Softwarebausteinen
- deren Beziehungen und
- Schnittstellen

- **Diagramme:**

- Komponentendiagramm
- Klassendiagramm (→ SWT)

UML Komponentendiagramm

Beschreibt Komponenten, deren Aufbau, Beziehungen und Schnittstellen

■ Hierarchie:

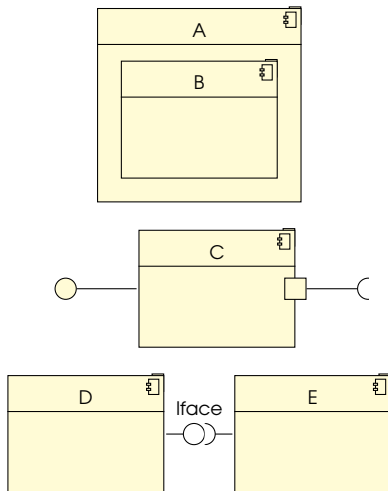
- Komponenten können durch Komponenten detailliert werden

■ Schnittstellen:

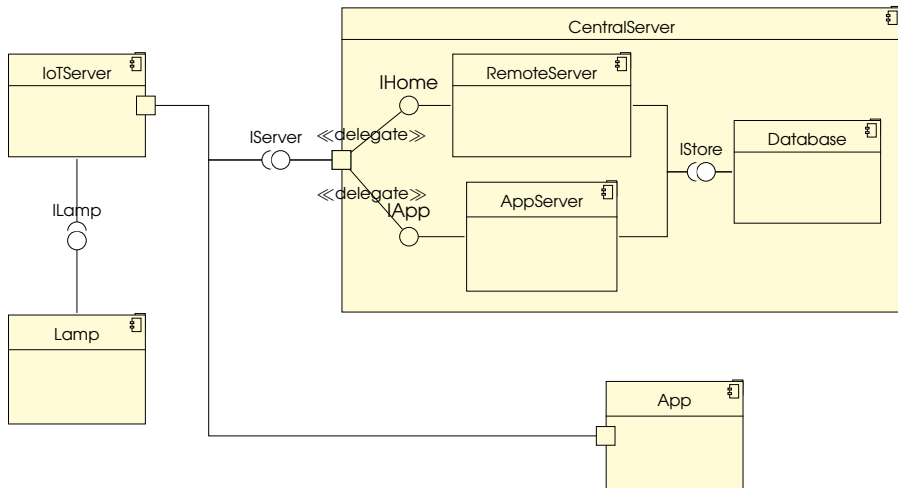
- Es gibt bereitgestellte und benötigte Schnittstellen
- Ports können Schnittstellen zusammenfassen

■ Komposition:

- Komponenten werden über passende Schnittstellen verbunden



Logical View: Beispiel



Development View (Sicht der Entwickler)

■ **Zeigt das System aus Sicht des Entwicklers:**

- Welche Module, IDE-Projekte, Code Repositories gibt es
- Wie wird das System aus diesen Projekten erzeugt

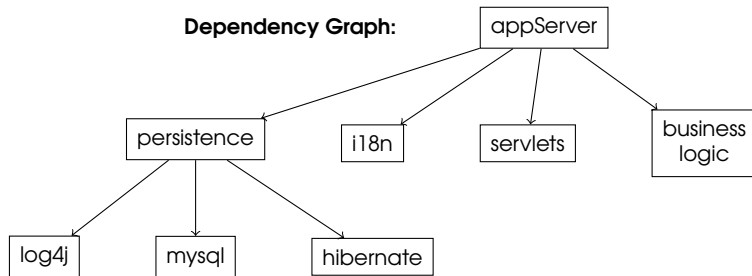
■ **Inhalt:**

- Codegenerierung: Welche Teile des Quellcodes werden generiert und aus welchen Artefakten?
- Build-Management: Wie wird das System aus seinem Quellcode erzeugt?
- Abhängigkeiten

■ **Diagramme:**

- Paketdiagramm
- Abhängigkeitsgraph (nicht UML)
- Build Dateien / Modul Meta-Informationen (nicht UML)

Development View: Beispiel



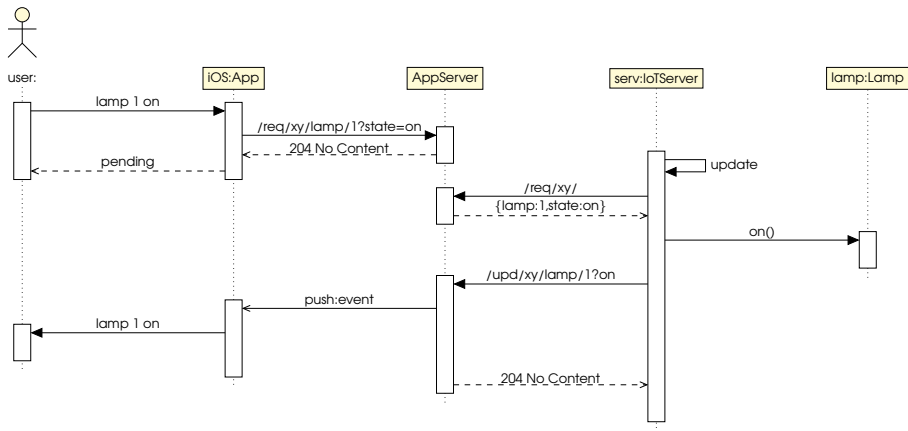
```
<groupId>gov.nasa</groupId>
<artifactId>jconstraints</artifactId>
<version>0.9.2-SNAPSHOT</version>
<packaging>jar</packaging>
<name>jConstraints</name>
<url>https://github.com/psycopaths/jconstraints</url>
<description>A library for modeling expressions and for interacting with constraint solvers</description>
```

Ausschnitt aus Maven POM

Process View (Dynamische Sicht)

- **Zeigt die Laufzeitsicht das Verhalten des Systems bzw. einzelner Bausteine zur Laufzeit:**
 - wie die Bausteine des Systems zur Laufzeit miteinander bzw. mit den Nachbarsystemen interagieren
 - wie Bausteine des Systems die wesentlichen Funktionen oder Anwendungsfälle erledigen
- **Inhalt:**
 - Wesentliche, beispielhafte Abläufe
- **Diagramme:**
 - Zustandsdiagramm (→ Kap. 3-Modellierung)
 - Aktivitätsdiagramm (→ SWT)
 - Sequenzdiagramm (→ SWT)

Process View: Beispiel



Physical View (Verteilung)

- **Zeigt die Verteilung von Systembestandteilen auf Hard- und Softwareumgebungen:**
 - welche Teile des Systems auf welchen Rechnern, an welchen geographischen Standorten oder in welchen Umgebungen ablaufen
- **Inhalt:**
 - Zuordnung von Artefakten zu Laufzeitumgebungen und Rechnern
 - Ggf. Zuordnung Artefakte / Komponenten
 - Kommunikationsverbindungen
- **Diagramme:**
 - Verteilungsdiagramm

UML Verteilungsdiagramm

Beschreibt Verteilung von Artefakten auf Geräte

■ Orte für Artefakte:

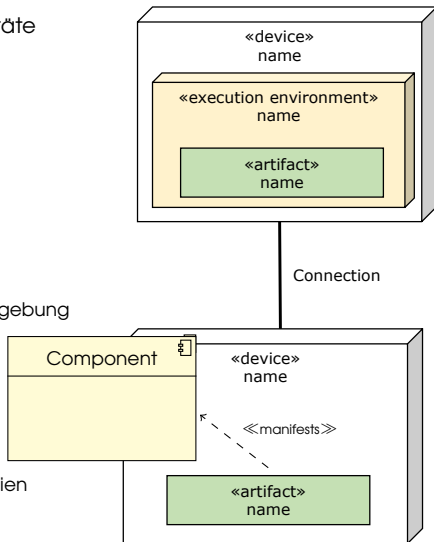
- Device: physikalisches Gerät
- Execution Environment: Logische Ausführungsumgebung

■ Artefakte:

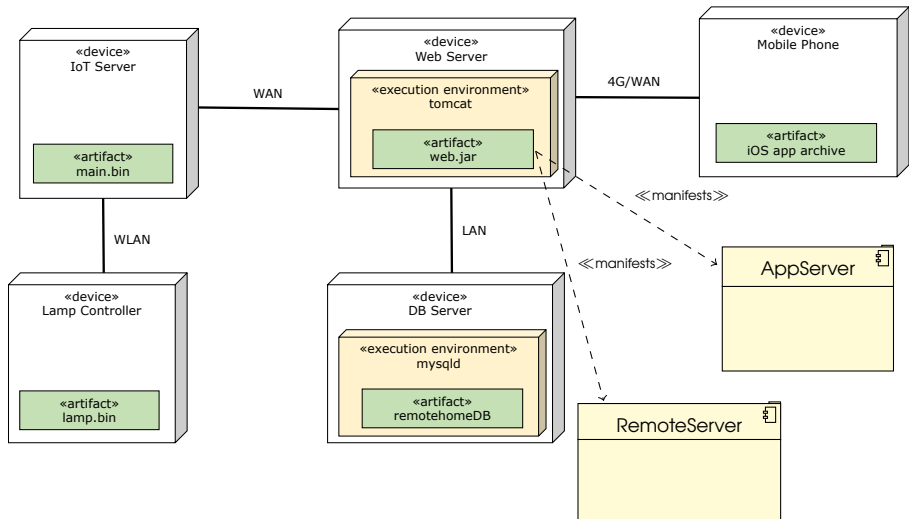
- Auslieferungsgegenstände der Software
- Laufen auf Gerät oder in Ausführungsumgebung
- Manifestieren Komponenten zur Laufzeit

■ Kommunikation:

- Zwischen Geräten
- Angabe konkreter Protokolle / Technologien möglich



Physical View: Beispiel



Agenda

1. Motivation
2. Begriffe und Definitionen
3. Treiber und Prinzipien
4. Dokumentation: 4+1 Sichten
5. Ausgewählte Architekturmuster
6. Qualität von Software Architektur
7. Zusammenfassung
8. Literatur und Standards

Ausgewählte Muster

- **Verteilung**
- Strukturierung
- Kommunikation
- Skalierbarkeit
- Entwicklung

Client-Server Architektur (Verteilung Explizit)

Eine **Client-Server Architektur** verteilt Anwendungslogik und -funktionalität auf eine Menge von Clients und Server Subsysteme, welche u.U. auf verschiedenen Maschinen ausgeführt werden und über Netzwerk kommunizieren.

Wichtige Eigenschaften:

- Verteilung der Daten einfach (explizit)
- Client muss andere Clients nicht kennen
- Unterteilt ein System in handhabbare Teile
- Unabhängiger Kontrollfluss (im Gegensatz zu verteilten Objekten)
- Server meistens verantwortlich für Persistenz und Konsistenz von Daten

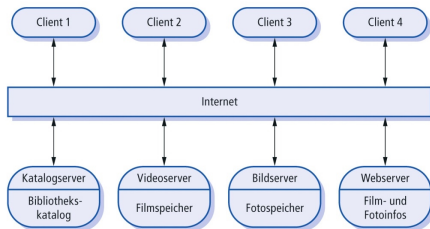
Client-Server Architektur

Herausforderungen:

- Client muss Identität des Servers kennen (zentrales Verzeichnis)
- Kein gemeinsames Datenmodell (Aufwand für Übersetzung)
- Definition der Kommunikation notwendig, z.B. RPC, SOAP, REST

Beispiele:

- Web-Services
- Datenbank



Objekt-orientierte Architektur (Verteilung Transparent)

Wichtige Eigenschaften:

- Kapselung (interne Datendarstellung von außen nicht sichtbar)
- Struktur: Menge von interagierenden Objekten
- Passt zu Objekt-orientiertem Design

Herausforderungen:

- Objekte müssen Identität anderer Objekte kennen, um zu interagieren
- Verteilung über Netzwerk muss möglich/transparent sein

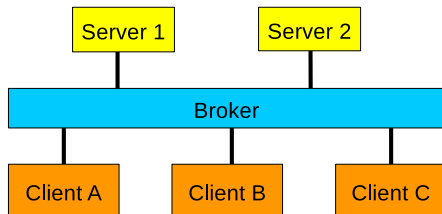
Beispiele:

- CORBA
- Enterprise Java Beans

Transparenz durch Object Broker / Proxies

Wichtige Eigenschaften:

- Broker als Indirektion zwischen Client und Server
- Clients müssen Server nicht mehr kennen
- Mehrere Broker und mehrere Server möglich
- Proxies implementieren Kommunikation mit nicht-lokalen Objekten



Herausforderungen:

- Broker sind Engpässe und Single-Point-of-Failure
- Verminderte Performanz durch Indirektion / Proxies

Ausgewählte Muster

- Verteilung
- **Strukturierung**
- Kommunikation
- Skalierbarkeit
- Entwicklung

Schichten (Horizontale Dekomposition)

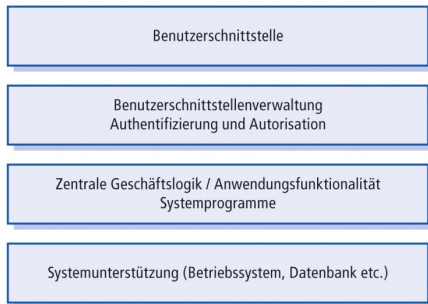
Schichtenarchitektur organisiert System in Menge von Schichten, wobei jede Schicht der darüber liegenden Schicht Funktionalität zur Verfügung stellt.

Beispiele:

- Betriebssysteme
- Kommunikationsprotokolle

Wichtige Eigenschaften:

- Element in einer Schicht nutzen nur:
 - andere Elemente in gleichen Schicht oder
 - Elemente aus der unmittelbar darunterliegenden Schicht



Schichten (Horizontale Dekomposition)

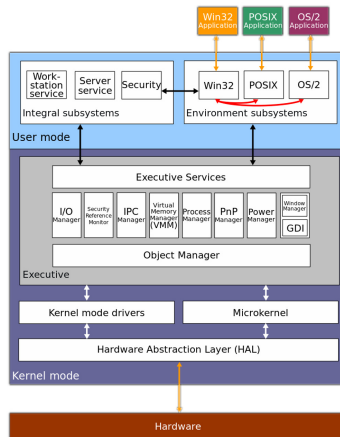
Beispiel: MS Windows 2000 OS

Eigenschaften (contd.):

- Erlauben Abstraktion von technischen Details in höheren Schichten
- Spezifische Aufgaben in Schichten (Anzeige, Logik, Persistenz)

Herausforderung:

- Oft schwierig saubere Schichttrennungen zu definieren



Quelle: MS Windows 2000 OS, [wikipedia.org](https://www.wikipedia.org)

Offene vs. Geschlossene Architektur

Geschlossene Architektur

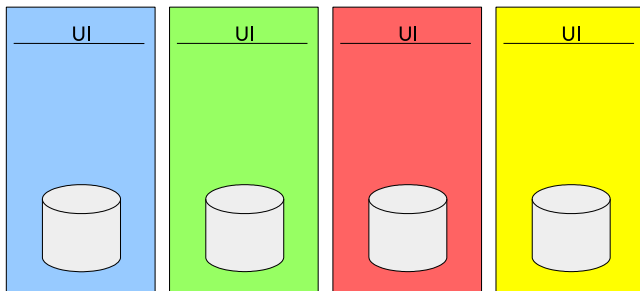
- Jede Schicht verwendet ausschließlich Dienste der direkt darunterliegenden Schicht
- Minimiert Abhängigkeiten zwischen den Schichten und reduziert Auswirkungen von Veränderungen

Offene Architektur

- Eine Ebene nutzt Dienste von darunterliegenden Schichten
- Vermeidet unnötige Fehlerquellen ("Durchschleifen" von Funktionen)
- Weicht Kapselung der Schichten auf und erhöht Abhängigkeiten

Partitionen (Vertikale Dekomposition)

Anstelle einer Anwendung, in der alle Features in **einen Prozess** gepackt werden, zerlege Anwendung von vornherein in **mehrere**, voneinander möglichst **unabhängige** Anwendungen.



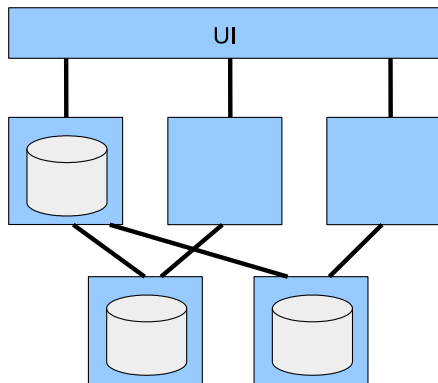
Kombination von Schichten und Partitionen

Schichten:

- Hierarchische Dekomposition
→ Baum von Subsystemen
- „Horizontale“ Unterteilung
- Offen vs. geschlossene Schichten

Partitionen:

- „Vertikale“ Unterteilung:
Jede Teilmenge behandelt eine Funktion
- Extremform der Entkoppelung:
semi-unabhängig



Pipes und Filter



Beispiele:

- Unix Pipes
- Compiler
- Regelungstechnische Systeme

Wichtige Eigenschaften:

- Filter müssen nichts über verbundene Elemente wissen
- Filter können parallel ausgeführt werden
- Jeder Filter arbeitet auf Datenstrom

Repositories

Beispiele:

- Datenbanken
- IDE

Eigenschaften:

- Unabhängige Agenten kollaborieren über zentrale Repräsentation der Daten
- Reduziert Notwendigkeit komplexe Daten zu duplizieren

Herausforderung:

- Synchronisation / Blackboard wird Engpass

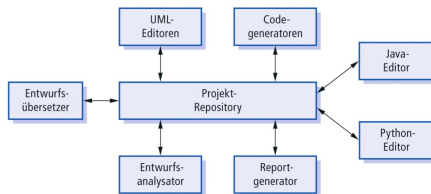


Abbildung 6.9: Eine Repository-Architektur für eine IDE.

Microservice Architektur

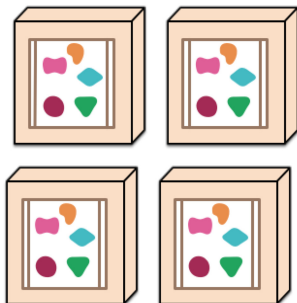
- In Microservice Architekturen sind Softwareapplikationen eine Menge von unabhängig einsetzbaren (deployable) Diensten
- **Wichtige Eigenschaften:**
 - Komponentenerlegung via Dienste leicht möglich
 - Organisiert um betriebswirtschaftliche Funktionen
 - Entwicklung eher Produkte als Projekte
 - Smarte Endpunkte und dumme Verbindungen
 - Ausfallsicher und Evolutionsfähig
 - Dezentrale Anwendungs- und Datenverwaltung

Microservice Architektur

A monolithic application puts all its functionality into a single process...



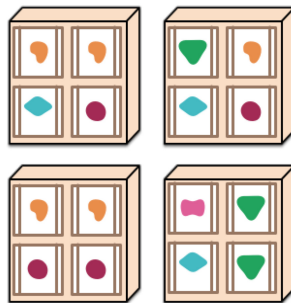
... and scales by replicating the monolith on multiple servers



A microservices architecture puts each element of functionality into a separate service...



... and scales by distributing these services across servers, replicating as needed.



Quelle: <http://martinfowler.com/articles/microservices.html>

Ausgewählte Muster

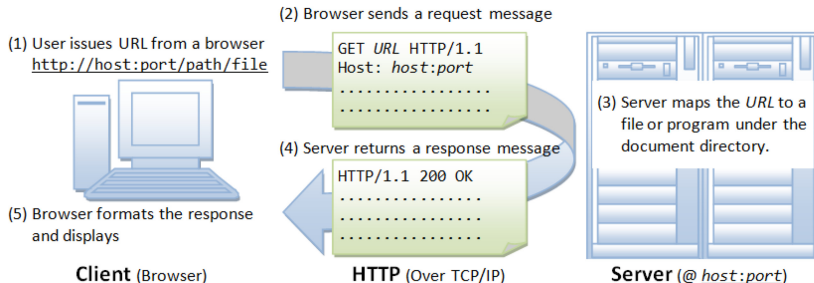
- Verteilung
- Strukturierung
- **Kommunikation**
- Skalierbarkeit
- Entwicklung

REST - Representational State Transfer

- REST-Paradigma von Roy Fielding 1994
als rationale Rekonstruktion des HTTP
Object Models: GET, POST, ...
- Einheitlicher Architekturstil für
Web-Services
- **Prinzipien:**
 - Client-Server
 - Zustandslosigkeit
 - Caching
 - Einheitliche Schnittstelle
 - Mehrschichtige Systeme
 - Code on Demand (optional)



HTTP als einheitliche Basis für REST



- Zustandsloses Protokoll
- Request Arten: GET, POST, HEAD, PUT, DELETE, ...
- Ursprünglich zur Auslieferung von Internetseiten
- Webserver kann Requests an beliebige Programme weitergeben

Beispiel: Google Drive

get	GET /files/ fileId	Gets a file's metadata by ID.
insert	POST https://www.googleapis.com/upload/drive/v2/files and POST /files	Insert a new file.
patch	PATCH /files/ fileId	Updates file metadata. This method supports patch semantics .
update	PUT https://www.googleapis.com/upload/drive/v2/files/ fileId and PUT /files/ fileId	Updates file metadata and/or content.
copy	POST /files/ fileId /copy	Creates a copy of the specified file.
delete	DELETE /files/ fileId	Permanently deletes a file by ID. Skips the trash. The currently authenticated user must own the file or be an organizer on the parent for Team Drive files.
list	GET /files	Lists the user's files.
touch	POST /files/ fileId /touch	Set the file's updated time to the current server time.
trash	POST /files/ fileId /trash	Moves a file to the trash. The currently authenticated user must own the file or be an organizer on the parent for Team Drive files.
untrash	POST /files/ fileId /untrash	Restores a file from the trash.
watch	POST /files/ fileId /watch	Start watching for changes to a file.
emptyTrash	DELETE /files/trash	Permanently deletes all of the user's trashed files.
generateIds	GET /files/generateIds	Generates a set of file IDs which can be provided in insert requests.
export	GET /files/ fileId /export	Exports a Google Doc to the requested MIME type and returns the exported content. Please note that the exported content is limited to 10MB.

Ereignis-basierte Architektur

Komponenten in **ereignis-basierter Architektur** liefern Dienste als Reaktion auf externe Ereignisse von anderen Komponenten.

■ Beispiele:

- Debugger (lauschen nach speziellen Breakpoints)
- Publish / Subscribe Systeme
- Graphische Benutzerschnittstellen
- Im **Broadcast Modell** werden Ereignisse an alle Subsysteme gesendet. Jedes Subsystem kann Ereignis behandeln.
- Im **Interrupt-Modell** wird Echtzeit-Interrupt von einem Handler aufgefangen und an andere Komponente weitergeleitet.

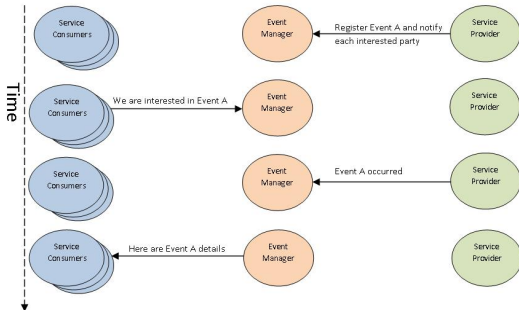
Ereignis-basierte Architektur

Wichtige Eigenschaften:

- Ereignisquelle muss Ereignissen nicht kennen (impliziter Aufruf)
- Unterstützt Wiederverwendung und Evolution

Herausforderungen:

- Komponenten haben keine Kontrolle von Berechnungsreihenfolge



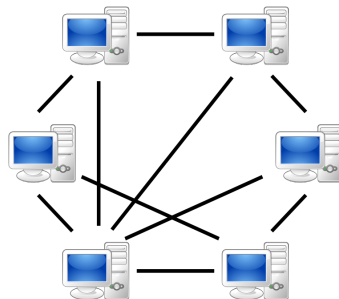
Peer to Peer

Wichtige Eigenschaften:

- Findet Peers durch Server oder Broadcast
- Kommuniziert im Anschluss nur mit Peers
- Reduziert Engpässe und robust gegenüber Ausfällen von Peers

Herausforderungen:

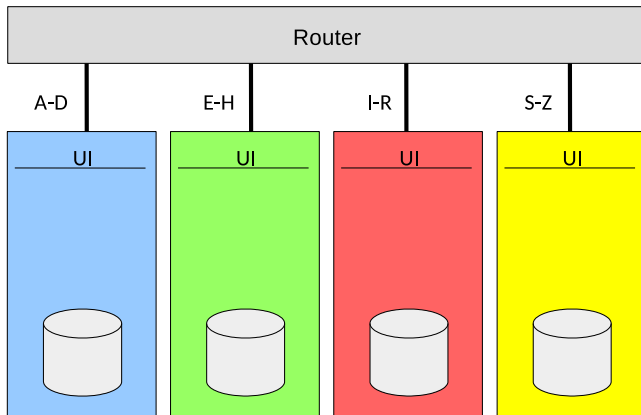
- Server kann Engpass werden
- Peers haben unvollständige Sicht – Synchronisation möglich



Ausgewählte Muster

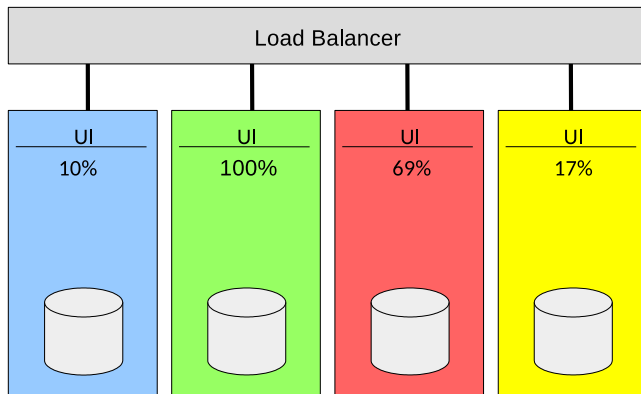
- Verteilung
- Strukturierung
- Kommunikation
- **Skalierbarkeit**
- Entwicklung

Sharding



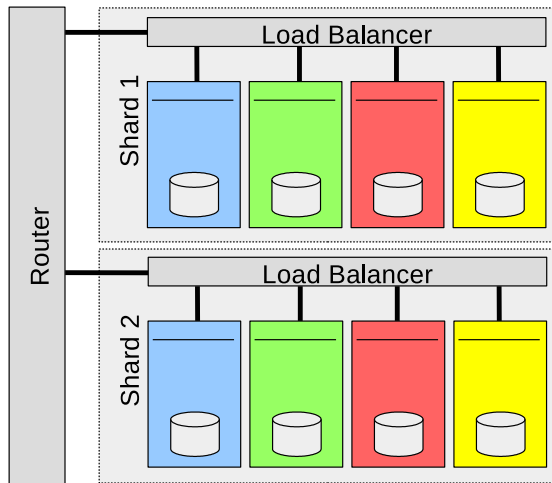
- Aufteilung der Eingaben auf dedizierte Prozesse
- **Beispiel:** Aufteilung nach Anfangsbuchstaben

Load Balancing



Anstatt Sharding mit festem Kriterium, z.B. Anfangsbuchstaben
→ dynamische Zuordnung der Bearbeitung, z.B. nach Auslastung

Kombinationen



Varianten können kombiniert werden um Skalierbarkeit zu erhöhen

Ausgewählte Muster

- Verteilung
- Strukturierung
- Kommunikation
- Skalierbarkeit
- **Entwicklung**

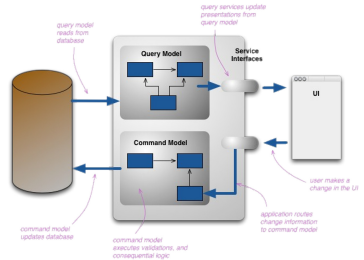
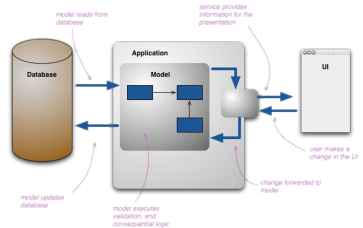
Command Query Responsibility Segregation

Problem:

Gemischtes Modell für Ansicht und Manipulation

Lösung:

Trennung von Sicht- und Manipulationsbefehlen



Quelle: <http://martinfowler.com/bliki/CQRS.html>

Semantische Versionierung

Versionierung mit Nummer **MAJOR.MINOR.PATCH**:

- **MAJOR** inkompatible API Änderungen
- **MINOR** neue Funktionalität aber backwards-compatible
- **PATCH** backwards-compatible Bug-fixes

Eigenschaften:

- Neuere PATCH und MINOR Versionen von Abhängigkeiten können einfach verwendet werden

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Treiber und Prinzipien
4. Dokumentation: 4+1 Sichten
5. Ausgewählte Architekturmuster
6. Qualität von Software Architektur
7. Zusammenfassung
8. Literatur und Standards

Wie unterstützt man geringe Abhängigkeit, geringe Auswirkung von Änderungen und hohe Wiederverwendbarkeit?

Kopplung (coupling)

Menge von Beziehungen zwischen Subsystemen bzw. Maß dafür, wie stark ein Element verbunden ist, Wissen über andere Elemente besitzt oder von anderen Elementen abhängt.

Typische Formen der Kopplung zwischen A und B:

- A besitzt Attribut mit Verweis auf Instanz von B
- A Objekt ruft Dienst eines B Objekts auf
- A hat Methode welche B Instanz referenziert
 - z.B. Parameter, lokale Variable, Rückgabewert
- A ist direkte oder indirekte Subklasse von B
- B ist Schnittstelle und A implementiert diese Schnittstelle

Starke vs. lose Kopplung

Starke Kopplung (schlecht)

- Lokale Änderungen notwendig bei Änderungen in verbundenen Klassen
- Isoliertes Verstehen der Klassen schwierig (big picture)
- Geringere Wiederverwendbarkeit, da Klassenverwendung Anwesenheit weiterer verbundener Klassen benötigt

Lose Kopplung (gut)

- Lose Kopplung = wenig Beziehungen zwischen Subsystemen
- Wenig Beziehungen $\Rightarrow$ geringe Auswirkung von Änderungen und hohe Wiederverwendbarkeit
- Einfache Wartung / lokale Änderung

Wie bündelt man ein Objekt, macht es verständlich, handhabbar und erreicht – ganz nebenbei – eine lose Kopplung?

Kohäsion (cohesion)

Menge von Beziehungen innerhalb eines Subsystems bzw. Maß dafür, wie stark zusammenhängend und gebündelt die Verantwortlichkeiten im Element sind.

- **Sehr niedrige Kohäsion:** Baustein bietet vollständig unterschiedliche Funktionalität
- **Niedrige Kohäsion:** Klasse hat eine Verantwortlichkeit für eine komplexe Aufgabe in einer komplexen Funktionalität, z.B. eine einzelne Schnittstelle für alle Datenbankzugriffe.
- **Mittlere Kohäsion:** Leichtgewichtige Klasse verantwortlich für einige wenige logische Bereiche, z.B. *Unternehmen* kennt Mitarbeiter und Finanzdetails.
- **Hohe Kohäsion:** Klasse mit moderater Verantwortlichkeit in einem funktionalen Bereich, welche mit anderen Klassen zusammenarbeitet.

Geringe vs. hohe Kohäsion

Geringe Kohäsion

Element mit geringer Kohäsion bietet verschiedene, unzusammenhängende Funktionalitäten. Solche Elemente sind unerwünscht und erzeugen folgende Probleme:

- schwer verständlich
- schwer wiederzuverwenden
- schwer zu warten
- anfällig, andauernd von Änderungen beeinflusst

Hohe Kohäsion

Element mit hohen verbundenen Verantwortlichkeiten. Um hohe Kohäsion aufzuweisen, muss eine Klasse:

- wenige Methoden besitzen,
- wenige Codezeilen groß sein,
- nicht zu viel Funktionalität bieten,
- eine hohe Abhängigkeit des Programmcodes besitzen.

Eigenschaften einer guten Architektur



■ Minimieren der Kopplung zwischen Modulen

- Ziel: Module müssen wenig über Interaktion mit anderen Modulen wissen
- Lose Kopplung erleichtert zukünftige Änderungen

■ Maximieren der Kohäsion in Modulen

- Ziel: Inhalte jeder Komponente sollten stark zusammenhängend sein
- Hohe Kohäsion erleichtert das Verstehen einer Komponente

Oft: Zielkonflikt!

Beispiel: Kopplung

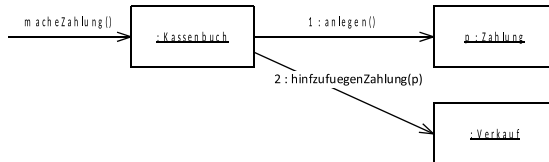
Gegeben seien folgende Klassen



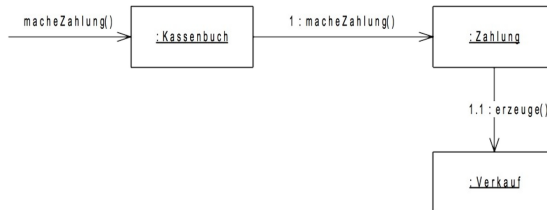
- Instanz „Zahlung“ muss erzeugt werden und mit Verkauf verbunden werden.
- Welche Klasse ist die Verantwortliche?

Mögliche Umsetzung

#1: Diese Lösung entspricht dem Erzeuger-Muster (creator pattern)



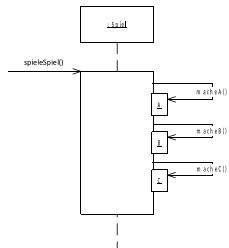
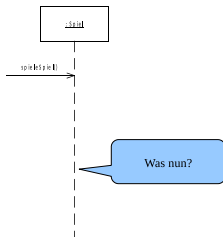
#2: Zahlung erzeugt Verkauf



Welche Lösung ist besser?

- Angenommen jeder *Verkauf* ist mit *Zahlung* verbunden.
- Lösung #1 besitzt Kopplung zwischen *Kassenbuch* und *Zahlung*, welche nicht in Lösung #2 vorkommt.
- Lösung #2 besitzt daher geringere Kopplung.

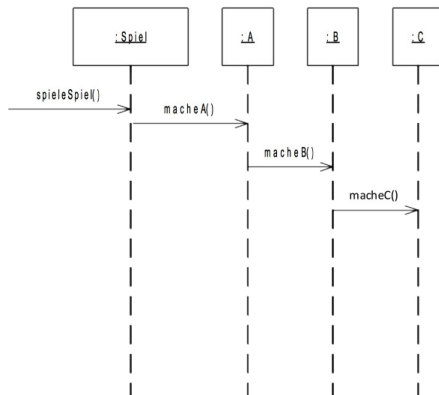
Beispiel: Kohäsion



Spiel macht alles.

Wie hängen
Aufgaben A, B und C
zusammen?

Falls geringe
Abhängigkeit, dann
auch geringe
Kohäsion!



A, B und C kapseln einzelne Aspekte $\Rightarrow$
hohe innere Kohäsion

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Treiber und Prinzipien
4. Dokumentation: 4+1 Sichten
5. Ausgewählte Architekturmuster
6. Qualität von Software Architektur
7. Zusammenfassung
8. Literatur und Standards

Zusammenfassung

Architektur

- Umsetzbarer Entwurf der eines Systems
- Bauplan

Dokumentation

- 4+1 Sichten

Qualität

- Kopplung
- Kohäsion

Vorgehen

- Treiber, Ziele, Prinzipien
- Herausforderungen

Muster

- Etablierte Lösungsmuster
- Adressieren Herausforderungen

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Treiber und Prinzipien
4. Dokumentation: 4+1 Sichten
5. Ausgewählte Architekturmuster
6. Qualität von Software Architektur
7. Zusammenfassung
8. Literatur und Standards

- Hruschka, Peter und Starke, Gernot. Software-Architektur kompakt: angemessen und zielorientiert, Spektrum, 2011.
<https://dx.doi.org/10.1007/978-3-8274-2835-6>
- Duggan, Dominic. Enterprise software architecture and design : entities, services and resources, Wiley, 2012.
<https://onlinelibrary.wiley.com/doi/pdf/10.1002/9781118180518>
- Fowler, Martin. Patterns of enterprise application architecture, Addison-Wesley, 2003.

Standards

- **ISO/IEC/IEEE 42010** Systems and software engineering — Architecture description
- **OMG 2007** OMG: Unified Modeling Language: Superstructure, Version 2.1.1. OMG document formal/2007-02-05.